CS5707 Artificial Intelligence
Lab 7. Image Processing

Yongmin Li*

We will use the Google Colaboratory (or Google Colab or simply Colab) in the labs of this module, which is a free Jupyter notebook environment hosted by the Google cloud services. The Colab uses the same format of Jupyter Notebook, a popular interactive development platform for Julia, Python and R.

Aims and Objectives

In this lab, we will work on the following problems in image processing by using a number of most influential methods over the past few years. We will use the "built" models only, and skip the processing of building these models. Hopefully from these tasks you will understand the underlying principles of these methods and their effectiveness in solving the relevant problems.

1. Frontal face detection using the Haar cascade detector.
2. Object detection from images using YOLO.
3. Instance segmentation of images using YOLO.

After this session, you should achieve the following learning objectives:

1. Implement a solution to detect faces from an input image using a pre-built model such as Haar cascade detector.
2. Implement a solution to perform object detection from images using a pre-built model such as YOLO.
3. Implement a solution to perform image segmentation using a pre-built model such as YOLO.
4. Evaluate the performance of the solutions developed.

Time Required

2 hours (with supervision).

Please note that the above number is only indicative and should be regarded as an average amount of time for all students. You may require significantly more or less time depending on personal circumstances, e.g. knowledge and understanding to the relevant subject topics, experience of the software used, and level of programming skills. Please do prepare to use more time outside the officially arranged sessions.

---

* First draft November 2020. These materials have been used for "CS5707 Artificial Intelligence" at Department of Compute Science, Brunel University London.

Image processing and computer vision has long been a popular (if not the most popular) research area in artificial intelligence. Actually, the recent resurgence of artificial intelligence was largely started by the redevelopment of neural networks (now commonly referred to as “deep networks”) on image recognition when AlexNet was introduced and won the IMAGENET Large Scale Visual Recognition Challenge (ILSVRC) in 2012.

You will have the opportunity to further study the machine learning and deep learning methods in another two modules of the MSc programme (Machine Learning and Deep Learning). In this lab, we will focus on the common problems of image processing and how to use the modern artificial intelligence techniques to solve these problems.

In this lab, we will use the following Python packages.

OpenCV is the most commonly used library for image processing and computer vision. Initially developed at Intel in 1999, it has seen its history of over 20 years. Being adopted, used and contributed by many researchers around the world, it now a cross-platform library of almost all the common image processing and computer vision function. Originally written in C/C++, it also has interfaces with Python, Java and Matlab. The website of the project can be found at

https://pypi.org/project/opencv-python/

The other packages are most wrappers of several other Python packages, especially for the image recognition and segmentation methods. Before you have the chance of studying these deep learning models in detail next year, it may be easier just to use these wrapper packages so you can complete a program with only a few lines of code. In this way, you would focus on understanding the problems and their solutions without getting confused by the complicated implementation details in the middle.

cvlib is for object detection. The project website is

https://www.cvlib.net/

Ultralytics YOLO is the latest generation models of YOLO as a unified framework supporting multiple vision tasks, including object detection, instance segmentation, pose estimation, image classification, and oriented bounding boxes.

https://docs.ultralytics.com/tasks/
https://docs.ultralytics.com/guides/instance-segmentation-and-tracking/

The data required in this lab will be much larger than in the previous labs. I have share a folder from Google Drive, so you can simply copy or link to your own Google Drive.

https://drive.google.com/drive/folders/1vf9INW8CBx7LH5R-YD2RS7uwqeAtBpjv?usp=sharing

Task 7.1 Face Detection

Face detection is normally the first stage of processing one has to perform for many image processing tasks with faces such as face recognition, face verification, facial expression recognition etc.

In this task, we will use the Haar cascade face detector to find faces from an input image. This method was first introduced by Viola and Jones in 2001. With the advance of modern machine learning technologies, many other models for face detection have been developed. But the Haar cascade detector is still one of the fastest detectors though it may not be the most accurate one.

First we install the OpenCV package and import it. We also need to import the patches for imshow() in OpenCV as it does not work properly within the Google Colab (but perfectly fine elsewhere) at the time of writing. You may find details of this issue from the internet and hopefully someone will fix the problem soon.

```
!pip install opencv-python
import cv2
from google.colab.patches import cv2_imshow
```

Next we download the trained model of the Haar cascade face detector. The model data file is also included in the shared folder I have provided. You may also load it from there if you wish.

```
!wget 'https://github.com/yongminli/data/raw/main/haarcascade_frontalface_d
efault.xml'
face_cascade = cv2.CascadeClassifier('haarcascade_frontalface_default.xml')
```

Now load one of your images into Colab. When you run this cell, Colab will show a button named “Choose Files”. Click the button and select the image files you would like to try – you may upload multiple files in one go. Make sure there are indeed human faces in the images, or if you want to see whether the detector fails on non-face images.

```
from google.colab import files
file = files.upload()
```

If you use the image I have, “brunel.jpg”, you may read the image using OpenCV’s imread() function, and convert it into graylevel using cvtColor().

```
img = cv2.imread('brunel.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

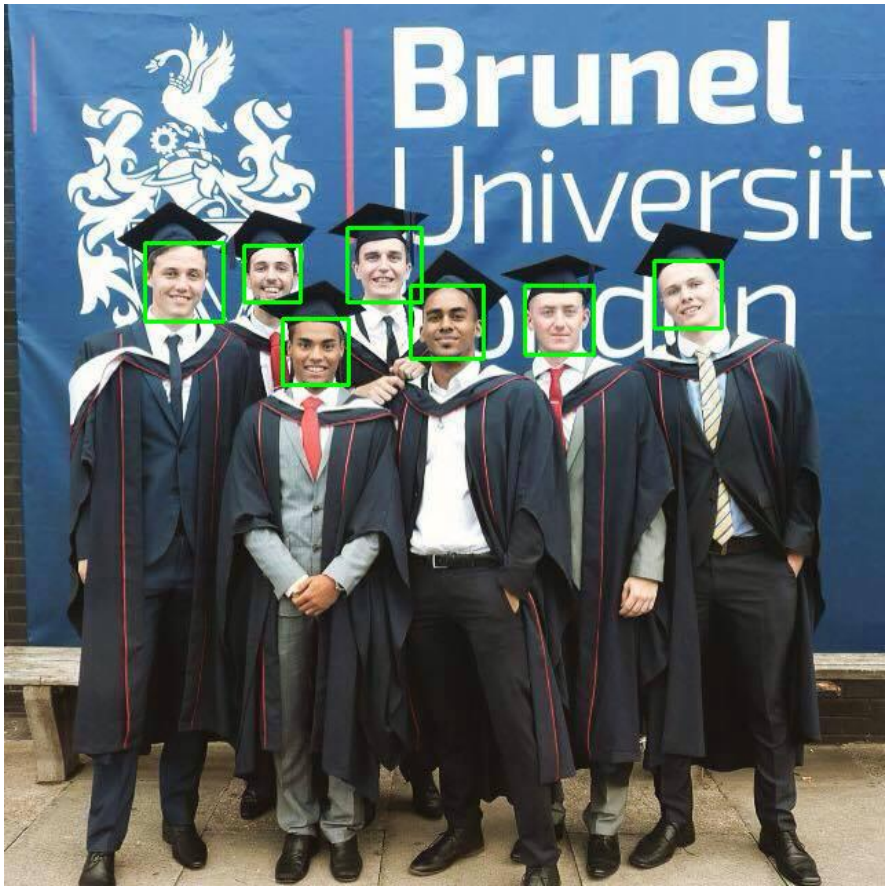
The detection is fairly straightforward by running the function detectMultiScale(). There are a few parameters you may use. Details can be found at the OpenCV

online documentation. We only use the minNeighbors here to avoid multiple detections too close to each other.

```
faces = face_cascade.detectMultiScale(image=gray,  minNeighbors=5)
```

Finally we can draw the detection results and show them on the image.

```
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (0, 255, 0), 2)
cv2_imshow(img)
```



Requirement for lab report:

1. Explain the process of face detection using the Haar cascade detector, in particular, on why Haar features can be effective to detect faces from images, and how the cascade of classifiers can save computation.
2. Complete the program and try a few of your own images with various conditions e.g. frontal/non-frontal faces, bright/dark lighting, non-face/animal faces etc.
3. Discuss your results and evaluate the performance.

Task 7.2 Object Recognition

Object recognition from images and videos has long been one of the most important topics in image processing and computer vision. With the modern AI technologies, we can comfortably use computer software to recognise thousands and tens of thousands of different object categories. For example, in the ILSVRC competition, there are 1000 different object categories and over 1.4 million images. More importantly, as we discussed in the lecture, it was this competition and its 2012 winner AlexNet inaugurated the new era of modern AI.

In this task, we will use YOLO, one of the best models for object recognition. Different to R-CNN, Fast R-CNN or Faster R-CNN where the scanning has to go for several rounds, YOLO only scans the input image once, therefore the name “You Only Look Once (YOLO)”. The project website of YOLO is

https://pjreddie.com/darknet/yolo/

where you can find details of the model and download the pre-built model files. There are two models provided: the default version (with name “yolov3”) and the tiny version (with name “yolov3-tiny”). The former is more accurate but slower, while the latter is less accurate but faster. The size of the tiny model is only 35MB but the default one is about 250MB.

You may load your models directly from the YOLO site, but it may take a while depending your network connection. Alternatively, these files are available in the folder I shared with you. It may be better to copy these models files into your Google Drive and mount it in your program.

As in the previous task, we use a simplified interface cvlib so that you can implement the program rather easily. Install the package and import it first.

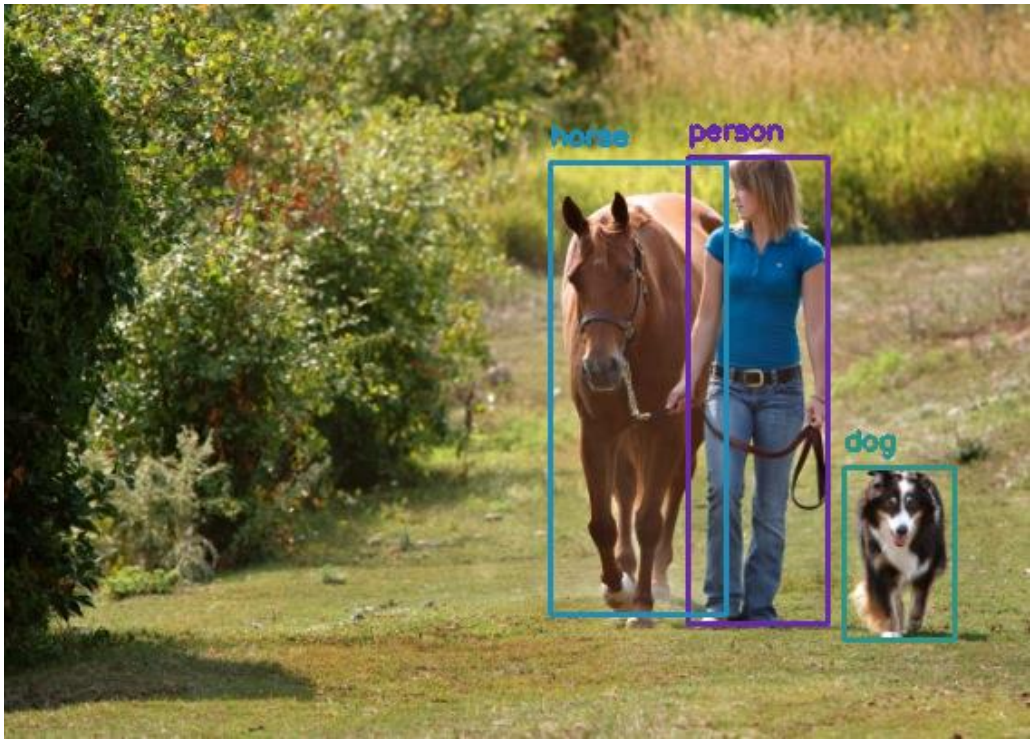
```
!pip install cvlib
import cvlib
from cvlib.object_detection import draw_bbox, YOLO
```

Download the model files, or mount the Google Drive to access the files locally (as instructed in the previous task). If we use the tiny version, we construct the model as follows.

```
config = path + 'yolov3-tiny.cfg'
weights = path + 'yolov3-tiny.weights'
labels = path + 'yolov3.txt'
yolo = YOLO(weights, config, labels)
```

Once the model is constructed, the detection and result displaying can be done with the following few lines of code.

```
img = cv2.imread(path+'objects.jpg')
bbox, label, conf = yolo.detect_objects(img)
yolo.draw_bbox(img, bbox, label, conf)
cv2_imshow(img)
```



Requirement for lab report:

1. Describe how you would perform object recognition from an input image using a pre-built model such as YOLO.
2. Complete the program.
3. Test both the default and tiny version of YOLO with a few images of your own choice.
4. Evaluate the performance and discuss your results.

Task 7.3 Image Segmentation

Image segmentation is to partition an image into a number of meaningful regions. It has many important applications such as medical imaging, object recognition, image and video compression, automated driving, to name a few.

Image segmentation is one of the well-studied subjects in image processing and computer vision. In recent years, the focus of research has been shifted to the deep learning based methods, the same way as in many other image processing problems.

In this task, we will use a pre-built YOLO segmentation model (yolov8m-seg.pt) to perform instance segmentation, i.e. a combination of object recognition and segmentation of each object instances along their boundaries.

We will use PixelLib, a high-level interface for image and video segmentation. First, install the package and import it together with other packages such as OpenCV.

```
!pip install ultralytics
from ultralytics import YOLO
import cv2
from google.colab.patches import cv2_imshow

# Load pretrained YOLOv8 segmentation model directly from ultralytics hub
model =
YOLO('https://github.com/ultralytics/assets/releases/download/v0.0.0/yolov8
m-seg.pt')
```

The yolov8m-seg.pt model is about 52MB, a moderate size for today's standard. You may load the model directly from the URL link, or download and save it to your Google Drive and load it from there locally. I have kept a copy of the model in the folder shared with you where you may copy or link to your own Drive. So you can mount your Google Drive and access the files directly just like a local drive.

```
from google.colab import drive
drive.mount('/content/drive')
path = "/content/drive/My Drive/data/"
```

In the above example, your data is saved in the "data" folder of your Google Drive. Choose an input image. In my example, I use the image "segment.jpg" that is available in the shared folder.

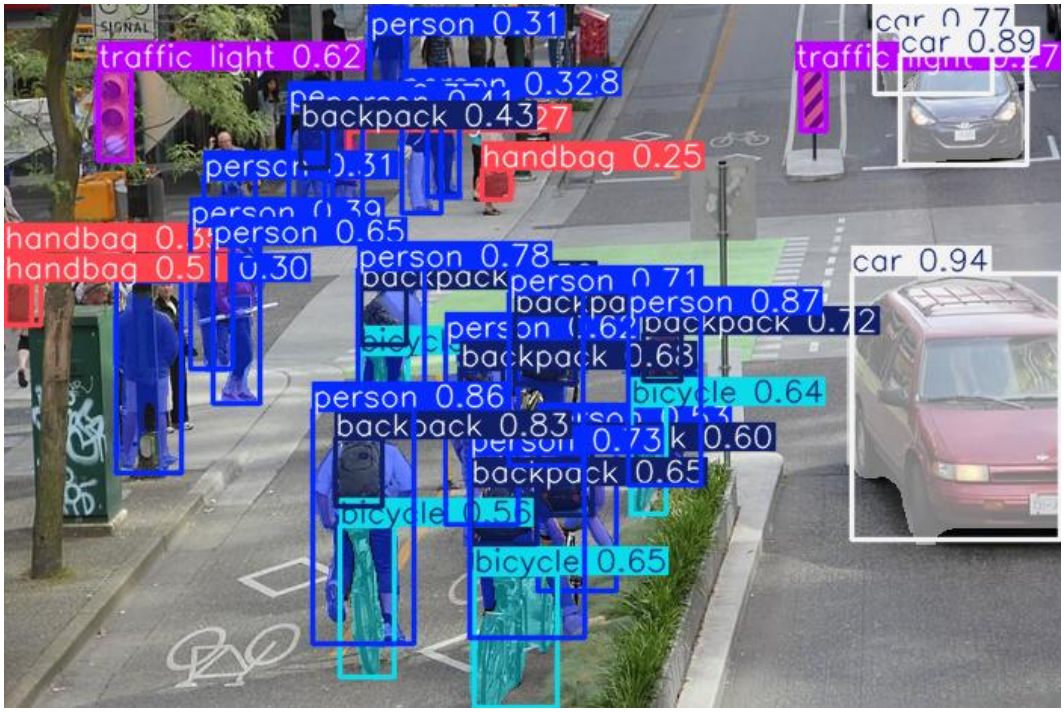
```
img = cv2.imread(path+'segment.jpg')
cv2_imshow(img)
```

The segmentation can be performed straightforwardly by calling the function model(img).

```
results = model(img)
annotated_img = results[0].plot()
cv2_imshow(annotated_img)
```

The input image and the segmentation results look like below.





Requirement for lab report:

1. Describe how you would perform image segmentation using a ResNet model.
2. Complete the program and test it with a few images.
3. Evaluate the performance and discuss your results.